

权重 Offloading

目 录

1 引言：当显存装不下整个模型	2
2 同步 Offloading：先搬后算	2
2.1 基本流程	2
2.2 正确性容易，性能不好	2
3 异步 Offloading：搬运与计算重叠	3
3.1 CUDA Stream：让搬运和计算并行	3
3.2 双缓冲：交替使用两块显存	3
3.3 CUDA Event：等待搬运完成	3
4 Pinned Memory：加速 H2D 传输	4
4.1 为什么普通内存慢	4
4.2 锁页内存	4
5 INT4 权重的整体管理	4
6 Offloading 粒度：常驻层与卸载层	5
6.1 全卸载不是唯一选择	5
6.2 常驻层 vs 卸载层	5
6.3 多层粒度	5
7 实战：估算延迟能否被隐藏	6
8 代码概念：双缓冲 + CUDA Event	6
9 本章你将学会	7
10 要点速查	7
11 小结	8

1 引言：当显存装不下整个模型

在 Lab5 的实验环境中，我们面对的是 H800 MIG 10G 配置，可用显存只有 10 GiB。Gemma4-12B 经过 GPTQ 量化到 INT4 后，模型权重约占 6 GiB。乍看 6 GiB 小于 10 GiB，似乎能装下，但推理时显存不只存放权重：KV Cache 随 batch size 和序列长度线性增长，激活值临时分配，再加上 CUDA context 和框架自身的开销，留给权重和 KV Cache 的空间捉襟见肘。

当 batch size 提到 8、最大序列长度设到 2048 时，KV Cache 可能占用数 GiB 显存。此时如果把全部 6 GiB 权重常驻 GPU，剩余空间根本不够存放 KV Cache，OOM 几乎不可避免。

直觉 不妨把 GPU 显存想象成一个不大的工作台。你要在上面同时摊开所有工具（权重）、正在写的草稿纸（KV Cache）和参考书（激活值）。工具太多，台面放不下。但你注意到一个关键事实：Transformer 是逐层执行的，计算第 i 层时，第 $i+1$ 层的工具完全用不到。何不把暂时不用的工具放回工具箱（CPU 内存），用到时再拿出来？

这就是 **Weight Offloading**（权重卸载）的核心思想：只让当前计算所需的权重常驻 GPU，其余权重保存在 CPU 内存中，执行到对应层之前再搬运到 GPU。本章我们从最简单的同步方案出发，逐步引入 CUDA Stream、双缓冲和 Pinned Memory，最终实现搬运与计算重叠的异步 Offloading。

2 同步 Offloading：先搬后算

2.1 基本流程

Transformer 的 48 个层按顺序执行，计算第 i 层时只用该层的权重。同步 Offloading 的做法很直接：在计算第 i 层之前，将该层权重从 CPU 搬运到 GPU 的一块缓冲区，等待搬运完成后执行该层的前向计算，然后进入下一层。

关键在于复用同一块 GPU 缓冲区。既然每一层在计算时独占使用自己的权重，计算完毕后该缓冲区就可以被下一层覆盖。我们只需要一块大小等于最大单层权重的 GPU 缓冲区，而不是全部 48 层的权重总和。

```
Layer 0: H2D(weight_0) → wait → compute(layer_0)
Layer 1: H2D(weight_1) → wait → compute(layer_1)
Layer 2: H2D(weight_2) → wait → compute(layer_2)
...
```

每一行内，H2D 传输和计算是串行的：传输完成才开始计算，计算完成才开始下一层的传输。时间线是一条没有重叠的直线。

2.2 正确性容易，性能不好

同步方案的优点是正确性几乎不证自明：每次计算前权重已经就位，不存在数据竞争。但缺点同样明显：GPU 在搬运权重时完全空闲，CPU 在 GPU 计算时无事可做，H2D 传输与计算完全串行。

设单层权重的 H2D 传输时间为 T_{H2D} ，单层计算时间为 T_{compute} ，则同步方案每层总耗时为：

$$T_{\text{sync}} = T_{\text{H2D}} + T_{\text{compute}} \quad (2.1)$$

如果 T_{H2D} 和 T_{compute} 相当，那么 GPU 有一半时间在等待数据，吞吐量直接减半。我们需要让搬运和计算同时进行。

3 异步 Offloading: 搬运与计算重叠

3.1 CUDA Stream: 让搬运和计算并行

GPU 上的操作被组织为 **Stream** (流)，同一个 Stream 内的操作按顺序执行，不同 Stream 之间可以并行。默认所有操作在 **default stream** (默认流) 上执行，它们之间是串行的。

要让 H2D 传输和计算同时进行，我们把它放到两个独立的 Stream 上：

1. **Compute Stream** (计算流)：执行层的前向计算
2. **Copy Stream** (搬运流)：执行 CPU 到 GPU 的权重传输

两个流由 GPU 硬件调度器并行执行，搬运第 $i+1$ 层权重的同时可以计算第 i 层。

在 PyTorch 中用 `torch.cuda.Stream()` 创建独立流，用 `with torch.cuda.stream(s): ...` 将操作指定到该流。跨流同步通过 `Event` 完成。

3.2 双缓冲: 交替使用两块显存

如果只有一块 GPU 缓冲区，搬运流写入缓冲区时，计算流也在读同一块缓冲区，会产生数据竞争。解决办法是准备两块大小相同的 GPU 缓冲区，记为 buffer A 和 buffer B，交替使用：

1. 计算第 i 层时，用 buffer A 存放第 i 层权重，同时在搬运流上把第 $i+1$ 层权重写入 buffer B
2. 计算第 $i+1$ 层时，用 buffer B 存放第 $i+1$ 层权重，同时在搬运流上把第 $i+2$ 层权重写入 buffer A
3. 如此交替，两块缓冲区轮流使用

这就是 **Double Buffering** (双缓冲) 的核心。计算流和搬运流各用一块缓冲区，互不干扰。

直觉 想象两个人在流水线上工作。A 负责加工零件，B 负责从仓库取零件。如果只有一个工作台，B 放零件时 A 没法加工，两人互相等待。给每个人一张工作台，B 把下一个零件放到 A 旁边的台子上，A 加工完手头的零件后直接从旁边的台子拿，B 则去取再下一个零件。两人各忙各的，流水不断。

3.3 CUDA Event: 等待搬运完成

双缓冲解决了缓冲区冲突，但还有一个问题：计算第 $i+1$ 层之前，必须确保第 $i+1$ 层的权重已经搬运到 buffer B。两个独立 Stream 之间没有隐式同步，需要显式等待。

CUDA Event (事件) 是 GPU 上的同步原语，用于在 Stream 之间传递信号。工作流程如下：

1. 搬运流完成第 $i+1$ 层传输后，记录一个 Event
2. 计算流在开始计算第 $i+1$ 层之前，调用 `wait_event` 等待该 Event
3. 如果搬运尚未完成，计算流自动阻塞；如果搬运已完成，立即继续

这样计算流既不会过早开始（读到旧数据），也不会过晚等待（搬运已完成就立即开工）。

时间线（异步双缓冲）：

```
Copy   | H2D(0) | H2D(1) | H2D(2) | H2D(3) | ...
Compute|       | Cmp(0) | Cmp(1) | Cmp(2) | ...
        ^wait  ^wait  ^wait
```

理想情况下，计算流几乎不用等待，每层耗时趋于 $\max(T_{\text{H2D}}, T_{\text{compute}})$ 而非两者之和。

4 Pinned Memory: 加速 H2D 传输

4.1 为什么普通内存慢

PyTorch 默认在可分页内存（pageable memory）上分配 CPU 张量。操作系统的虚拟内存机制允许将不常用的内存页换出到磁盘，因此 GPU DMA 引擎无法直接访问可分页内存，它不知道某一页下一刻是否还在物理内存中。

实际传输时，驱动程序先在内部分配一块临时锁定内存，把数据从可分页内存拷贝到锁定内存，再由 DMA 引擎从锁定内存传到 GPU。这就多了一次 CPU 侧的拷贝，拖慢了 H2D 传输速度。

4.2 锁页内存

Pinned Memory（锁页内存）是被操作系统标记为不可换出的内存页，始终驻留在物理 RAM 中。GPU DMA 引擎可以直接读取它，省去中间拷贝步骤。

在 PyTorch 中，用 `torch.Tensor.pin_memory()` 将张量锁定到页内存：

```
weight_cpu = weight_cpu.pin_memory()
```

这行代码返回一个新的张量，其底层存储被锁定在物理内存中。之后从该张量向 GPU 传输数据时，DMA 直接读取，速度显著提升。

Pinned Memory 会占用物理内存且不可换出，分配过多会挤压系统其他进程的可用内存。*Lab5* 中只需为模型权重分配 *pinned memory*，不必把所有 CPU 张量都锁定。

直觉 把可分页内存比作图书馆里随时可能被别人借走的书。你想复印一本书，但不知道下一刻它还在不在架上，只能先借出来放到自己桌上（临时锁定内存），再复印。*Pinned Memory* 则是把书锁在保险柜里，复印机直接对着保险柜扫描，一步到位。

5 INT4 权重的整体管理

INT4 量化后，模型权重不再是一个连续的浮点张量，而是三个紧密耦合的部分：

1. **qweight** (量化权重): 打包后的 INT4 数据, 每两个 4-bit 值打包进一个 uint8 字节
2. **scales** (缩放因子): 每个 group 一组, FP16 精度
3. **zeros** (零点): 每个 group 一组, 对称量化时为 None, 非对称量化时为 FP16

这三者必须作为一个整体搬运。如果只搬 qweight 而漏掉 scales, 反量化时会用错误的缩放因子, 导致输出完全错误。在实现 Offloading 时, 我们把每一层的 qweight、scales、zeros 打包到一个列表或字典中, 一次性 H2D 传输到 GPU 的对应缓冲区。

```
layer_weights_cpu[i] = {
    "qweight": qw.pin_memory(),
    "scales":   sc.pin_memory(),
    "zeros":    zr.pin_memory() if zr is not None else None,
}
bufs[i % 2]["qweight"].copy_(layer_weights_cpu[i]["qweight"],
                              non_blocking=True)
bufs[i % 2]["scales"].copy_(layer_weights_cpu[i]["scales"],
                             non_blocking=True)
```

第一行把第 i 层的三个张量收集到字典中并锁页; 第二、三行用 `non_blocking=True` 异步拷贝到 GPU 缓冲区。`non_blocking=True` 配合独立 Stream 使用时, 拷贝立即返回, 不阻塞 CPU 线程。

6 Offloading 粒度: 常驻层与卸载层

6.1 全卸载不是唯一选择

把全部 48 层权重都放到 CPU、每次计算前再搬运, 是最极端的方案。它最大化地节省了 GPU 显存, 但每一层都引入 H2D 传输开销。如果 GPU 显存虽然装不下全部 48 层, 但能装下一部分, 我们可以把一些层常驻 GPU, 其余层卸载到 CPU。

6.2 常驻层 vs 卸载层

将 48 层分为两组:

1. **Resident Layers** (常驻层): 权重始终在 GPU 上, 无需搬运。通常是模型靠前的层 (如 embedding 层和前几层), 或者计算量大、搬运不划算的层。
2. **Offloaded Layers** (卸载层): 权重保存在 CPU, 计算前搬运到 GPU。通常是靠后的多数层。

设常驻 R 层、卸载 $L - R$ 层, 常驻权重占 M_R 显存。选择 R 的原则是: 常驻层占用显存后, 剩余显存恰好够存放 KV Cache 和激活值, 同时卸载层的搬运延迟尽可能被计算覆盖。

6.3 多层粒度

Offloading 的粒度不一定是单层。可以把多个连续层打包为一个组, 整组搬运整组计算。增大粒度的优势是减少 Event 同步次数和 Python 调用开销; 劣势是缓冲区更大, 可能无法全部常驻。在 Lab5 中, 48 层的权重结构相似, 按单层粒度卸载是最自然的选择。

7 实战：估算延迟能否被隐藏

我们来算一笔账，判断在 H800 MIG 10G 上异步 Offloading 是否能有效隐藏延迟。

例 取 H800 MIG 10G 配置。Gemma4-12B 的 INT4 权重总量约 6 GiB，分 48 层，平均每层权重约 $\frac{6144}{48} = 128$ MiB。PCIe Gen4 x16 的实测 H2D 带宽约 32 GB/s，换算为 $32 \times 1024 = 32\,768$ MiB/s。

单层权重的 H2D 传输时间：

$$T_{\text{H2D}} = \frac{128}{32} \times 768 \approx 0.0039 \text{ s} \approx 3.9 \text{ ms} \quad (7.1)$$

考虑两种场景：

Decode 阶段 (batch_size=8，每次生成 1 个 token)：单层计算量很小，矩阵-向量乘占主导，实测单层计算时间约 2 ms。由于 $T_{\text{H2D}} = 3.9 \text{ ms} > T_{\text{compute}} = 2 \text{ ms}$ ，搬运成为瓶颈，异步方案每层耗时趋于 $\max(3.9, 2) = 3.9 \text{ ms}$ ，无法完全隐藏。

Prefill 阶段 (batch_size=8，prompt 长度 512)：矩阵-矩阵乘，计算量大幅增加，实测单层计算时间约 8 ms。由于 $T_{\text{H2D}} = 3.9 \text{ ms} < T_{\text{compute}} = 8 \text{ ms}$ ，搬运被完全隐藏，异步方案每层耗时趋于 8 ms，与全量加载几乎无差别。

结论：decode 阶段 Offloading 会引入约 50% 的延迟开销，prefill 阶段几乎无开销。增大 batch size 可以提升 decode 的 T_{compute} ，使搬运被更好隐藏。

直觉 关键洞察：Offloading 的性能取决于搬运时间和计算时间的比值。当计算时间大于搬运时间时（prefill 或大 batch），Offloading 几乎免费；当搬运时间大于计算时间时（小 batch decode），Offloading 成为瓶颈。这就是为什么提高 batch size 不仅均摊了算子发射开销，还能让 Offloading 的代价趋于零。

8 代码概念：双缓冲 + CUDA Event

下面用 Python 伪代码展示异步 Offloading 的核心逻辑。实际实现中，run_layer 函数执行该层的前向计算，weight_cpu 列表存放各层的 pinned memory 权重。

```
import torch

class AsyncOffloader:
    def __init__(self, weights_cpu, num_layers):
        self.weights_cpu = weights_cpu
        self.num_layers = num_layers
        self.bufs = [
            {k: torch.empty_like(v, device="cuda")
             for k, v in weights_cpu[0].items()},
            {k: torch.empty_like(v, device="cuda")
             for k, v in weights_cpu[0].items()},
        ]
        self.copy_stream = torch.cuda.Stream()
        self.events = [torch.cuda.Event() for _ in range(num_layers)]
```

```

self._prefetch(0, 0)

def _prefetch(self, layer_idx, buf_idx):
    with self.copy_stream:
        for k, v in self.weights_cpu[layer_idx].items():
            if v is not None:
                self.bufs[buf_idx][k].copy_(v, non_blocking=True)
        self.events[layer_idx].record(self.copy_stream)

def forward(self, run_layer):
    for i in range(self.num_layers):
        self.events[i].wait()
        buf = self.bufs[i % 2]
        if i + 1 < self.num_layers:
            self._prefetch(i + 1, (i + 1) % 2)
        run_layer(i, buf)

```

逐行说明：__init__ 中分配两块 GPU 缓冲区 bufs[0] 和 bufs[1]，创建独立的 copy_stream 和每层一个 event，最后调用 _prefetch(0, 0) 预取第 0 层到 buffer 0。_prefetch 在 copy_stream 上执行异步拷贝，完成后记录 Event。forward 是主循环：先 wait 确保当前层权重就位，然后启动下一层预取，最后执行当前层计算。计算和预取在不同 Stream 上并行，实现重叠。

9 本章你将学会

1. 解释为什么 INT4 量化后 6 GiB 权重仍然可能超出 10 GiB 显存限制
2. 实现同步 Offloading 并理解其 H2D 传输与计算串行的瓶颈
3. 用 CUDA Stream 和双缓冲实现异步 Offloading，使搬运与计算重叠
4. 用 CUDA Event 在跨 Stream 之间同步，确保权重就位后再计算
5. 用 pin_memory() 加速 H2D 传输，理解锁页内存的原理和代价
6. 将 INT4 的 qweight、scales、zeros 作为整体搬运管理
7. 估算 H2D 传输时间与计算时间的关系，判断延迟能否被隐藏
8. 选择合适的 Offloading 粒度，平衡显存节省与搬运开销

10 要点速查

要点	说明
同步 Offloading	H2D 与计算串行，每层 $T_{\text{H2D}} + T_{\text{compute}}$
异步 Offloading	独立 Stream 双缓冲，每层 $\max(T_{\text{H2D}}, T_{\text{compute}})$
CUDA Stream	搬运流与计算流分离，硬件并行调度
双缓冲	两块 GPU 缓冲区交替使用，避免读写冲突
CUDA Event	跨 Stream 同步原语，搬运完成后通知计算流
Pinned Memory	锁页内存，DMA 直传，省去中间拷贝
INT4 权重	qweight + scales + zeros 整体搬运
常驻层 vs 卸载层	部分层常驻 GPU，其余卸载到 CPU

延迟隐藏条件	$T_{\text{compute}} > T_{\text{H2D}}$ 时搬运被隐藏
--------	--

11 小结

当显存无法容纳全部模型权重时，Weight Offloading 是最直接的应对策略。同步方案将 H2D 传输与计算串行执行，正确性容易保证但性能损失大。异步方案通过 CUDA Stream 分离搬运流与计算流，用双缓冲避免冲突，用 CUDA Event 保证同步，使每层耗时从 $T_{\text{H2D}} + T_{\text{compute}}$ 降为 $\max(T_{\text{H2D}}, T_{\text{compute}})$ 。

在 Gemma4-12B 的实际场景中，prefill 阶段计算量大，Offloading 几乎免费；decode 阶段计算量小，搬运可能成为瓶颈，需要通过增大 batch size 来提升计算时间以隐藏传输延迟。Pinned Memory 是加速 H2D 传输的基础设施，INT4 的三件套权重需要整体管理。这些技巧组合在一起，让我们在 10 GiB 显存的约束下实现了更大的 batch size，为后续的算子优化和调度优化打下基础。

讲义基于 HPC Lab5 实验指导编写